Apache Airflow Assessment 1

Employee Salary Processing

Objective

The objective of this assessment is to build an Apache Airflow DAG to process employee salary data using PythonOperator and file handling concepts.

The workflow performs:

- Create employee file
- Read employee data
- Calculate total salary expense
- Find highest salary
- Generate salary report

DAG Code

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


# Task 1 - Create Employee File
def create_employee_file():

    data = """Rahul,45000
Sneha,52000
Amit,61000
Priya,47000
Kiran,39000"""

    with open("/tmp/employees.txt", "w") as file:
        file.write(data)

    print("Employee file created successfully")
```

```python
# Task 2 - Read Employee Data
def read_employee_data():

    with open("/tmp/employees.txt", "r") as file:
        data = file.readlines()

    for line in data:
        print(line.strip())


# Task 3 - Calculate Salary Expense
def calculate_salary_expense():

    total_salary = 0

    with open("/tmp/employees.txt", "r") as file:
        data = file.readlines()

    for line in data:

        salary = int(line.strip().split(",")[1])

        total_salary = total_salary + salary

    print(f"Total Salary Expense = {total_salary}")


# Bonus Task - Find Highest Salary
def find_highest_salary():

    highest_salary = 0
    employee_name = ""

    with open("/tmp/employees.txt", "r") as file:
        data = file.readlines()

    for line in data:

        name, salary = line.strip().split(",")

        salary = int(salary)
```

```
if salary > highest_salary:

            highest_salary = salary

            employee_name = name

    print(f"Highest Salary = {highest_salary}")

    print(f"Employee = {employee_name}")


# Task 4 - Generate Salary Report
def generate_salary_report():

    report = """Employee Salary Report
Total Employees = 5
Total Salary Expense = 244000
Status = Processed Successfully"""

    with open("/tmp/salary_report.txt", "w") as file:
        file.write(report)

    print("Salary report generated successfully")


# DAG Definition
with DAG(

    dag_id="employee_salary_processing",

    start_date=datetime(2025, 1, 1),

    schedule="@daily",

    catchup=False,

    tags=["training"]

) as dag:


    task1 = PythonOperator(
        task_id="create_employee_file",
```

```
        python_callable=create_employee_file
    )


    task2 = PythonOperator(
        task_id="read_employee_data",

        python_callable=read_employee_data
    )


    task3 = PythonOperator(
        task_id="calculate_salary_expense",

        python_callable=calculate_salary_expense
    )


    task4 = PythonOperator(
        task_id="find_highest_salary",

        python_callable=find_highest_salary
    )


    task5 = PythonOperator(
        task_id="generate_salary_report",

        python_callable=generate_salary_report
    )


    task1 >> task2 >> task3 >> task4 >> task5
```

Press:

CTRL + D

to save the DAG file.